

## LAB SESSION 02

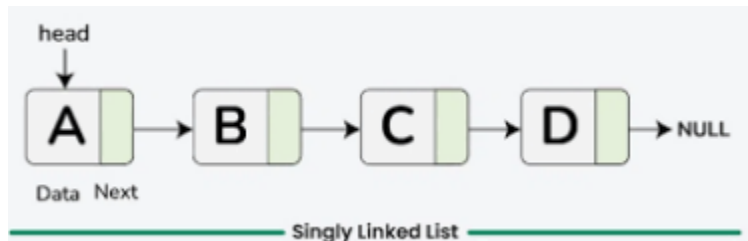
### Singly Linked List Operations

#### THEORY

A singly linked list is a dynamic linear data structure made up of nodes, where each node contains:

- Data (the actual value)
- A pointer (or reference) to the next node in the list

Unlike arrays, linked lists do not use contiguous memory. Each node is allocated memory separately and is linked using pointers. This provides flexibility in memory usage and makes insertion and deletion more efficient compared to arrays, especially in scenarios where memory needs to be allocated/deallocated frequently.



Structure of a Node:

```
struct Node {  
    int data;  
    Node* next;  
};
```

Key Operations:

- Insertion – At the beginning, middle, or end
- Deletion – Remove a node from the list
- Traversal – Visit each node and perform an action (e.g., print)
- Search – Find a node containing a specific value

Advantages:

- Dynamic size – Can grow/shrink as needed
- Efficient insertion/deletion – Especially at the beginning

Disadvantages:

- ☐ No random access –  $O(n)$  time to access elements
- ☐ More memory usage – Extra pointer in each node
- ☐ Sequential access only

Applications:

- ☐ Implementing stacks and queues
- ☐ Dynamic memory management
- ☐ Adjacency lists in graph algorithms
- ☐ Polynomial arithmetic

## PROCEDURE

1. Define a Node structure containing data and next.
2. Create multiple nodes and link them using pointers.
3. Traverse the linked list to display its elements.
4. Implement insertion (at beginning or end) and deletion operations.
5. Compile and run the code to validate output.

## CODE

```
#include <iostream>

using namespace std;

struct Node {
    int data;
    Node* next;
};

void printList(Node* head) {
    Node* temp = head;
    while (temp != NULL) {
        cout << temp->data << " -> ";
```

```

temp = temp->next;
}
cout << "NULL" << endl;
}

int main() {
// Create nodes
Node* head = new Node();
Node* second = new Node();
Node* third = new Node();
// Assign data and links
head->data = 10;
head->next = second;
second->data = 20;
second->next = third;
third->data = 30;
third->next = NULL;
// Print the linked list
printList(head);
return 0;
}

```

EXPECTED OUTCOME

10 -> 20 -> 30 -> NULL

### ALGORITHM FOR TRAVERSING A LINKED LIST

Step 1: [INITIALIZE] SET PTR = START

Step 2: Repeat Steps 3 and 4 while PTR != NULL

Step 3: Apply Process to PTR

-->DATA

Step 4: SET PTR = PTR

-->NEXT

[END OF LOOP]

Step 5: EXIT

Step 1: Initialize

Set PTR = START

This means **PTR points to the first node** of the linked list.

Step 2: Loop Condition

Repeat the next steps **while PTR is not NULL**

If PTR becomes NULL, it means we have reached the **end of the list**.

Step 3: Process the Data

Apply process to PTR → DATA

This usually means **displaying or using the data** stored in the current node.

Step 4: Move to the Next Node

Set PTR = PTR → NEXT

Move PTR to the **next node** in the list.

Step 5: Exit

When PTR becomes NULL, **stop the traversal**.

If the linked list is:

10 → 20 → 30 → NULL

Traversal will print:

10

20

30 Start from the first node, visit each node, process its data, and move to the next node until the list ends.

```
void display(struct node *ptr) {  
    while(ptr != NULL) {  
        printf("%d ", ptr->data); // process  
        ptr = ptr->next;  
    }  
}
```

```
} Call as display(START);
```

display() Function (Linked List)

This function is used to **print (display) all elements of a linked list.**

#### Function Definition

```
void display(struct node *ptr)
```

The function receives a pointer ptr

ptr initially points to the **first node (START)** of the linked list

While Loop

```
while(ptr != NULL)
```

The loop runs **until ptr becomes NULL**

NULL means **end of the linked list**

Printing Data (Processing)

```
printf("%d ", ptr->data);
```

This prints the **data stored in the current node**

This is the **process step** of traversal

Move to Next Node

```
ptr = ptr->next;
```

Moves ptr to the **next node** in the linked list

End of Function

When ptr becomes NULL, the loop stops

All nodes have been displayed

#### Function Call

```
display(START);
```

START is the pointer to the **first node**

The traversal begins from the **beginning of the list**

#### Simple Example

If the linked list is:

5 → 10 → 15 → NULL

**Output will be:**

5 10 15

The function starts from the first node, prints each node's data, moves to the next node, and stops at NULL.

### **C++ CODE**

```
#include <iostream>
using namespace std;

// Define a node of the linked list
struct Node {
    int data;
    Node* next;
};

// Function to traverse and print the linked list
void traverse(Node* start) {
    Node* ptr = start; // Step 1: Initialize PTR = START

    // Step 2: Repeat while PTR != NULL
    while (ptr != NULL) {
        cout << ptr->data << " "; // Step 3: Process PTR->DATA
        ptr = ptr->next;           // Step 4: PTR = PTR->NEXT
    }
    cout << endl;
}
```

```
// Utility function to add a new node at the end  
void insertEnd(Node*& start, int value) {
```

```
    Node* newNode = new Node();  
    newNode->data = value;  
    newNode->next = NULL;
```

```
    if (start == NULL) {  
        start = newNode;  
        return;  
    }
```

```
    Node* temp = start;  
    while (temp->next != NULL) {  
        temp = temp->next;  
    }  
    temp->next = newNode;  
}
```

```
// Main function
```

```
int main() {  
    Node* start = NULL;
```

```
    // Insert some nodes  
    insertEnd(start, 10);  
    insertEnd(start, 20);  
    insertEnd(start, 30);  
    insertEnd(start, 40);
```

```
    cout << "Traversing the linked list: ";  
    traverse(start);
```

```
    return 0;  
}
```

### ALGORITHM: INSERT A NEW NODE IN THE BEGINNING OF THE LINKED LIST

Step 1: IF AVAIL = NULL, then

Write OVERFLOW

Go to Step 7

[END OF IF]

Step 2: SET New\_Node = AVAIL

Step 3: SET AVAIL = AVAIL->NEXT

Step 4: SET New\_Node->DATA = VAL

Step 5: SET New\_Node->Next = START

Step 6: SET START = New\_Node

Step 7: EXIT

This algorithm is used to **add a new node at the start (beginning)** of a linked list.

Take a free node

Put data in it

Connect it before the first node

Make it the new START

Step 1: Check for Free Memory

If AVAIL = NULL

This means **no free memory is available**

Write **OVERFLOW** and stop

Step 2: Get a Free Node

Set New\_Node = AVAIL

Take one free node from the available list

Step 3: Update AVAIL

Set AVAIL = AVAIL → NEXT

Move AVAIL to the next free node

Step 4: Store Data

Set New\_Node → DATA = VAL

Store the given value VAL in the new node

Step 5: Link New Node to List

Set New\_Node → NEXT = START

New node points to the old first node



Step 6: Update START

Set  $START = New\_Node$

New node becomes the first node of the list

Step 7: Exit

Insertion is complete

This algorithm inserts a new node at the beginning by allocating a free node, storing data, linking it to the first node, and updating START.

START

↓

$10 \rightarrow 20 \rightarrow 30 \rightarrow NULL$

We want to **insert value 5 at the beginning**.

Step 1:  $AVAIL \neq NULL$

Memory is available

Step 2:

$New\_Node = AVAIL$

A free node is taken

Step 3:

$AVAIL = AVAIL \rightarrow NEXT$

Move to next free node

Step 4:

$New\_Node \rightarrow DATA = 5$

Store value 5 in new node

Step 5:

$New\_Node \rightarrow NEXT = START$

New node points to old first node (10)

Step 6:

$START = New\_Node$

New node becomes the first node

START

↓

$5 \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow NULL$

```
#include <iostream>
using namespace std;

// Define node structure
struct Node {
    int data;
    Node* next;
};

// Function to insert a node at the beginning
void insertAtBeginning(Node*& start, int val) {
    // Step 1: Create a new node
    Node* newNode = new(nothrow) Node; // nothrow prevents crash if memory fails

    if (newNode == NULL) { // Step 1 check (Overflow)
        cout << "OVERFLOW: No memory available!" << endl;
        return;
    }

    // Step 2: Assign value to new node
    newNode->data = val;

    // Step 3: Point new node to current start
    newNode->next = start;

    // Step 4: Update start
    start = newNode;
}
```

```
// Function to traverse and display linked list
void traverse(Node* start) {
    Node* ptr = start;
    while (ptr != NULL) {
        cout << ptr->data << " ";
        ptr = ptr->next;
    }
    cout << endl;
}

// Main function
int main() {
    Node* start = NULL; // Initially empty list

    // Insert nodes at the beginning
    insertAtBeginning(start, 30);
    insertAtBeginning(start, 20);
    insertAtBeginning(start, 10);

    cout << "Linked List after insertions: ";
    traverse(start); // Expected Output: 10 20 30

    return 0;
}
```

## ALGORITHM TO INSERT A NEW NODE AT THE END OF THE LINKED LIST

Step 1: IF AVAIL = NULL, then

Write OVERFLOW

Go to Step 10

[END OF IF]

Step 2: SET New\_Node = AVAIL

Step 3: SET AVAIL = AVAIL->NEXT

Step 4: SET New\_Node->DATA = VAL

Step 5: SET New\_Node->Next = NULL

Step 6: SET PTR = START

Step 7: Repeat Step 8 while PTR->NEXT != NULL

Step 8: SET PTR = PTR ->NEXT

[END OF LOOP]

Step 9: SET PTR->NEXT = New\_Node

Step 10: EXIT

This algorithm is used to **add a new node at the last position** of a linked list.

Create a new node

Move to the last node

Attach the new node at the end

Step 1: Check for Free Memory

If AVAIL = NULL

No free node is available

Write **OVERFLOW** and stop

Step 2: Get a Free Node

Set New\_Node = AVAIL

Take one free node from memory

Step 3: Update AVAIL

Set AVAIL = AVAIL → NEXT

Move AVAIL to the next free node

Step 4: Store the Data

Set New\_Node  $\rightarrow$  DATA = VAL

Store the value to be inserted

Step 5: Set Next as NULL

Set New\_Node  $\rightarrow$  NEXT = NULL

Because this will be the **last node**

Step 6: Start from First Node

Set PTR = START

Pointer starts from the beginning

Step 7 & 8: Traverse to Last Node

Repeat moving

PTR = PTR  $\rightarrow$  NEXT

Until PTR  $\rightarrow$  NEXT = NULL

This means **PTR is at the last node**

Step 9: Attach New Node

Set PTR  $\rightarrow$  NEXT = New\_Node

New node is linked at the end

Step 10: Exit

Insertion is complete

This algorithm inserts a new node at the end by allocating memory, traversing to the last node, and linking the new node.

## Initial Linked List

START

↓

10  $\rightarrow$  20  $\rightarrow$  30  $\rightarrow$  NULL

We want to **insert value 40 at the end.**

**Step 1:** AVAIL  $\neq$  NULL

Memory is available

**Step 2:**

New\_Node = AVAIL

A free node is taken

**Step 3:**

AVAIL = AVAIL  $\rightarrow$  NEXT

Move to next free node

**Step 4:**

New\_Node  $\rightarrow$  DATA = 40

Store value 40

**Step 5:**

New\_Node  $\rightarrow$  NEXT = NULL

This will be the last node

**Step 6:**

PTR = START

Pointer starts from first node (10)

**Step 7 & 8:**

Move PTR until PTR  $\rightarrow$  NEXT = NULL

PTR stops at node **30**

**Step 9:**

PTR  $\rightarrow$  NEXT = New\_Node

Attach new node at the end

**Linked List After Insertion**

START

↓

10  $\rightarrow$  20  $\rightarrow$  30  $\rightarrow$  40  $\rightarrow$  NULL

**Output (Traversal)**

10 20 30 40

**ALGORITHM TO INSERT A NEW NODE AFTER A NODE THAT HAS VALUE NUM**

Step 1: IF AVAIL = NULL, then

Write OVERFLOW

Go to Step 12

[END OF IF]

Step 2: SET New\_Node = AVAIL

Step 3: SET AVAIL = AVAIL  $\rightarrow$  NEXT

Step 4: SET New\_Node  $\rightarrow$  DATA = VAL

Step 5: SET PTR = START

Step 6: SET PREPTR = PTR

Step 7: Repeat Steps 8 and 9 while PREPTR->DATA != NUM

Step 8: SET PREPTR = PTR

Step 9: SET PTR = PTR->NEXT

[END OF LOOP]

Step 10: PREPTR->NEXT = New\_Node

Step 11: SET New\_Node->NEXT = PTR

Step 12: EXIT

This algorithm is used to **insert a new node immediately after the node whose data value is NUM.**

Find the node with value NUM

Insert the new node right after it

Adjust links properly

Step 1: Check Free Memory

If AVAIL = NULL

No free node is available

Write **OVERFLOW** and stop

Step 2: Get a Free Node

Set New\_Node = AVAIL

Take a free node from memory

Step 3: Update AVAIL

Set AVAIL = AVAIL → NEXT

Move AVAIL to the next free node

Step 4: Store Data

Set New\_Node → DATA = VAL

Store the new value to be inserted

Step 5: Start from First Node

Set PTR = START

PTR points to the first node

Step 6: Initialize PREPTR

Set PREPTR = PTR

PREPTR is used to keep track of the node with value NUM

Step 7, 8, 9: Search for NUM

Repeat until PREPTR → DATA = NUM

Move both pointers forward

- PREPTR = PTR
- PTR = PTR → NEXT

This finds the node **after which insertion is required**

Example: Insert a Node AFTER a Node with Value NUM

Initial Linked List

START → 10 → 20 → 30 → NULL

Task: Insert value 25 **after node with value 20**.

Step-by-Step Working

Step 1: AVAIL ≠ NULL Memory available

Step 2: New\_Node = AVAIL → Take a free node

Step 3: AVAIL = AVAIL → NEXT → Update free list

Step 4: New\_Node → DATA = 25 → Store data

Step 5: PTR = START → Start from first node

Step 6: PREPTR = PTR → Initialize PREPTR

Step 7, 8, 9: Search for NUM

- PREPTR → DATA = 10 → Not 20 → Move: PREPTR = PTR (10), PTR = PTR → NEXT (20)
- PREPTR → DATA = 20 → Found!

Step 10: PREPTR → NEXT = New\_Node → Link new node after 20

Step 11: New\_Node → NEXT = PTR → Link new node to next node (30)

Step 12: Exit

Linked List After Insertion

START → 10 → 20 → 25 → 30 → NULL

Traversal Output

10 20 25 30



Find the node with value 20, insert 25 after it, adjust links.

Step 10: Link New Node

Set PREPTR → NEXT = New\_Node

New node is linked after the node with value NUM

Step 11: Complete the Link

Set New\_Node → NEXT = PTR

New node now points to the next node

Step 12: Exit

Insertion is complete

This algorithm inserts a new node after a specified value by searching the node, linking the new node, and updating pointers.

```
#include <iostream>
using namespace std;

// Node structure
struct Node {
    int data;
    Node* next;
};

// Function to get a node from AVAIL
Node* getNode(Node*& AVAIL) {
    if (AVAIL == NULL) {
        cout << "OVERFLOW" << endl;
        return NULL;
    }
    Node* newNode = AVAIL;
    AVAIL = AVAIL->next; // move AVAIL forward
    newNode->next = NULL;
    return newNode;
}
```

```
// Function to insert after a node with value NUM
void insertAfter(Node*& START, Node*& AVAIL, int VAL, int NUM) {
    // Step 1-3: Take node from AVAIL
    Node* newNode = getNode(AVAIL);
    if (newNode == NULL) return; // no free nodes

    // Step 4: Assign data
    newNode->data = VAL;

    // Step 5-6: Traverse to find NUM
    Node* PTR = START;
    while (PTR != NULL && PTR->data != NUM) {
        PTR = PTR->next;
    }

    // Step 7: If NUM not found
    if (PTR == NULL) {
        cout << "Node with value " << NUM << " not found." << endl;
        // Return node back to AVAIL
        newNode->next = AVAIL;
        AVAIL = newNode;
        return;
    }
}
```

```
// Step 8-9: Insert after NUM
newNode->next = PTR->next;
PTR->next = newNode;
}
```

```
// Function to display linked list
void display(Node* start) {
    Node* temp = start;
    while (temp != NULL) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}
```

```
// Function to initialize AVAIL with free nodes
Node* initializeAvail(int n) {
    Node* head = NULL;
    for (int i = 0; i < n; i++) {
        Node* temp = new Node;
        temp->next = head;
        head = temp;
    }
    return head;
}
```

```

// Main function
int main() {
    Node* START = NULL;
    Node* AVAIL = initializeAvail(10); // pool of 10 free nodes

    // Manually build a linked list: 10 -> 20 -> 30
    Node* n1 = getNode(AVAIL);
    n1->data = 10;
    START = n1;

    Node* n2 = getNode(AVAIL);
    n2->data = 20;
    n1->next = n2;

    Node* n3 = getNode(AVAIL);
    n3->data = 30;
    n2->next = n3;
    n3->next = NULL;

    cout << "Original List: ";
    display(START);

    // Insert 25 after 20
    insertAfter(START, AVAIL, 25, 20);

    cout << "After inserting 25 after 20: ";
    display(START);

    // Try inserting after non-existing node
    insertAfter(START, AVAIL, 40, 50);

    cout << "Final List: ";
    display(START);

    return 0;
}

```

**TASKS:**

1. Write a function to insert a new node at the beginning.
2. Count the total number of nodes in the linked list.
3. Write a function to search for a value in the linked list.